Lecture 5 · Machine-Level Programming II: Control

10/8 (목) · 10/13 (녹화) · 슬라이드 "Lecture 5 Control" (전체) + 10/8 수업 녹취 + 10/13 녹화 강의 (조건 코드 ~ switch)

1 제어 흐름과 프로세서 상태

왜 control이 필요한가

- 기본 동작: 명령어를 읽고 실행한 뒤 %rip를 그 명령어 크기만큼 증가 → 다음 명령어로 순차 진행 (소스 코드를 한 줄씩 진행하는 것과 같음)
- 반복문 끝에서 조건이 만족되면 루프 처음으로 되돌아가야 함 → jump
- 조건에 따라 다른 분기로 가야 함 → 이를 처리하는 것이 control flow
- 이번 강의 주제: 조건 코드 → 조건 분기 → 반복문 → switch문: C의 if / loop / switch가 기계어 수준에서 어떻게 구현되는지

Processor State (x86-64, 일부)

- %rax, %rbx, … %r15: 범용 레지스터, 임시 데이터 저장 · %rsp: runtime stack의 top 위치
- **%rip**: instruction pointer: 다음에 실행할 명령어 주소 (general purpose 16개와 별개)
- **CF ZF SF OF**: condition codes, 최근 연산 결과의 상태

> **핵심: condition code + %rip = 조건 분기**
> 산술 연산이나 비교 명령은 조건 코드(CF, ZF, SF, OF)를 설정한다. 조건부 점프 명령은 이 조건 코드의 조합을 검사하여 점프 여부를 결정하고, 실제 점프 목적지는 명령어에 지정된 레이블이나 주소로 정해진다. 따라서 조건 코드는 분기 여부를 판단하는 기준이고, 점프 명령어는 제어 흐름의 목적지를 지정한다. 루프 안의 점프, switch 상태 이동이 모두 이 조합으로 구현된다.

2 Condition Codes

네 가지 flag (모두 1-bit 레지스터)

flag	이름	set 조건	주 해석
CF	Carry Flag	MSB에서 carry out (뺄셈이면 borrow)	**unsigned** overflow
ZF	Zero Flag	결과가 0	공통
SF	Sign Flag	결과의 MSB가 1 (signed로 음수)	**signed**
OF	Overflow Flag	two's complement overflow	**signed**

Implicit setting(산술 명령의 부수효과)

- add, sub 등 flag를 갱신하는 산술 명령은 실행 결과에 따라 조건 코드를 갱신 (side effect) — 명령어마다 갱신하는 flag가 다름 (inc/dec는 CF 보존). 예: addq src, dst ↔ $t = a + b$
  - CF: MSB에서 carry out → unsigned overflow (결과가 해당 크기에 담기지 않는 경우)
  - ZF: $t = 0$이면 set · SF: $t < 0$(signed, MSB=1)이면 set
  - OF: two's complement overflow: a, b가 양수인데 t가 음수, 또는 a, b가 음수인데 t가 0 또는 양수

> 덧셈의 OF: $OF = (a < 0 = b < 0) \ \wedge \ (t < 0 \neq a < 0)$. 부호가 다른 두 값(양수+음수)을 더하면 overflow는 절대 발생하지 않음

- **leaq는 flag를 바꾸지 않음** (주소 계산 전용)

> **해석은 사용자의 몫**
>
> 하드웨어는 덧셈이 signed인지 unsigned인지 모름 → CF와 OF를 **둘 다** 계산해서 세팅. unsigned 덧셈으로 볼지 signed 덧셈으로 볼지는 프로그래머·컴파일러의 해석. unsigned로 보면 CF를, signed로 보면 OF를 본다. 같은 flag 비트가 상황에 따라 두 가지 의미로 쓰인다는 것을 기억할 것.

손계산 1 · 8-bit 덧셈의 flag

연산	결과 (hex → 값)	CF	ZF	SF	OF	해석
100 + 50	0x96 → −106	0	0	1	1	양+양=음 → signed overflow
200 + 100 (unsigned)	0x2C → 44	1	0	0	0	300 > 255 → unsigned overflow
(−100) + (−100)	0x38 → 56	1	0	0	1	음+음=양 → signed overflow
255 + 1 (= −1 + 1)	0x00 → 0	1	1	0	0	unsigned로는 overflow, signed로는 정상

마지막 줄: 같은 비트 연산인데 unsigned면 overflow(CF=1), signed면 −1 + 1 = 0으로 정상(OF=0) → 해석의 차이.

cmp와 test: flag를 명시적으로 세팅

cmp (compare)

- `cmpq src2, src1`: cmpq b, a는 $a - b$를 계산하되 **destination에 저장하지 않음**. 레지스터는 갱신하지 않고 flag 세팅만 목적
- **피연산자 순서 주의: 두 번째 operand − 첫 번째 operand**. `cmpq %rsi, %rdi` → %rdi − %rsi
- CF: $a < b$(unsigned)면 borrow 발생 → set · ZF: $a - b = 0$, 즉 $a = b$면 set · SF: $a - b < 0$(signed)이면 set — 정확히는 뺄셈 비트 결과의 MSB (signed overflow 시 실제 대소와 어긋남, 뒤 예시 참조) · OF: two's complement overflow 시 set

> 뺄셈의 OF: $OF = (a < 0 \neq b < 0) \ \wedge \ (t < 0 \neq a < 0)$

- 뺄셈에서의 CF: a − b는 내부적으로 $a + (-b)$로 계산 → 실제로 MSB에서 carry out이 나면 CF = 0, 안 나면 CF = 1. carry out이 안 났다 = 위에서 빌려왔다(borrow) = unsigned로 $a < b$. 즉 CF는 "실제 carry 비트"가 아니라 "borrow가 있었는가".

8bit 예시	a + (~b+1)	carry out	CF	의미
3 − 2	00000011 + 11111110	있음	0	3 ≥ 2 (unsigned)
1 − 2	00000001 + 11111110	없음	1	1 < 2 (unsigned), 결과 11111111 = −1

test

- `testq src2, src1` : testq b, a는 $a \ \& \ b$를 계산하되 저장하지 않음
- operand 하나를 mask로 두면 유용. 특정 비트만 1인 마스크를 주면 a의 그 비트가 1인지 ZF로 판단
- ZF: $(a \ \& \ b) = 0$이면 set · SF: $(a \ \& \ b) < 0$이면 set (MSB=1) · **CF, OF는 항상 clear(0)**. AND 연산에는 carry/overflow 개념이 없음
- 대표 용법: `testq %rax, %rax` : 자기 자신과 AND → 결과 = 원래 값 그대로 → ZF=1이면 %rax가 0, SF=1이면 음수 (0인지 음수인지 한 번에 확인)
- 정리: flag는 산술 연산으로도, cmp/test로도 세팅됨. 대부분 두 값을 비교하는 용도. cmp로 $a - b$의 부호·overflow·0 여부 확인, test로 0/음수 확인.

x86-64 명령어 실행기 — Machine-Level Execution

예제 프로그램(Preset) 산술: OF와 CF

한 단계 | 되돌리기 | 자동 재생 | 리셋

PC=0 · 실행 횟수=0 · 실행 가능

명령어 흐름 — Instruction Trace

```
00  movabsq $0x7fffffffffffffff, %rax
01  addq $1, %rax
02  movq $-1, %rbx
03  addq $1, %rbx
04  subq $1, %rbx
05  negq %rbx
06  notq %rbx
```

레지스터 — 64-bit Registers

레지스터	Hex	Signed decimal
%rax	0x0000000000000000	0
%rcx	0x0000000000000000	0
%rdx	0x0000000000000000	0
%rbx	0x0000000000000000	0
%rsp	0x0000000000001100	4352
%rbp	0x0000000000001100	4352
%rsi	0x0000000000000000	0
%rdi	0x0000000000000000	0

조건 코드 — Condition Codes

플래그	현재 값	의미
ZF	0	Zero Flag
SF	0	Sign Flag
CF	0	Carry / Borrow Flag
OF	0	Overflow Flag

스택 메모리 — 256 Bytes

주소	낮은 주소 → 높은 주소 (8바이트)
0x0000000000001000	00 00 00 00 00 00 00 00
0x0000000000001008	00 00 00 00 00 00 00 00
0x0000000000001010	00 00 00 00 00 00 00 00
0x0000000000001018	00 00 00 00 00 00 00 00
0x0000000000001020	00 00 00 00 00 00 00 00

주소	낮은 주소 → 높은 주소 (8바이트)
0x0000000000001028	00 00 00 00 00 00 00 00
0x0000000000001030	00 00 00 00 00 00 00 00
0x0000000000001038	00 00 00 00 00 00 00 00
0x0000000000001040	00 00 00 00 00 00 00 00
0x0000000000001048	00 00 00 00 00 00 00 00
0x0000000000001050	00 00 00 00 00 00 00 00
0x0000000000001058	00 00 00 00 00 00 00 00
0x0000000000001060	00 00 00 00 00 00 00 00
0x0000000000001068	00 00 00 00 00 00 00 00
0x0000000000001070	00 00 00 00 00 00 00 00
0x0000000000001078	00 00 00 00 00 00 00 00
0x0000000000001080	00 00 00 00 00 00 00 00
0x0000000000001088	00 00 00 00 00 00 00 00
0x0000000000001090	00 00 00 00 00 00 00 00
0x0000000000001098	00 00 00 00 00 00 00 00
0x00000000000010A0	00 00 00 00 00 00 00 00
0x00000000000010A8	00 00 00 00 00 00 00 00
0x00000000000010B0	00 00 00 00 00 00 00 00
0x00000000000010B8	00 00 00 00 00 00 00 00
0x00000000000010C0	00 00 00 00 00 00 00 00
0x00000000000010C8	00 00 00 00 00 00 00 00
0x00000000000010D0	00 00 00 00 00 00 00 00
0x00000000000010D8	00 00 00 00 00 00 00 00
0x00000000000010E0	00 00 00 00 00 00 00 00
0x00000000000010E8	00 00 00 00 00 00 00 00
0x00000000000010F0	00 00 00 00 00 00 00 00

주소	낮은 주소 → 높은 주소 (8바이트)
0x00000000000010F8	00 00 00 00 00 00 00 00

```
예제 프로그램을 선택한 뒤 한 단계씩 실행합니다.

초기 ZF/SF/CF/OF=0은 교육용 초기값입니다.
64비트 산술 immediate는 부호 확장된 imm32를 사용합니다.
pushq immediate도 signed imm32를 64비트로 부호 확장합니다.
0xFFFFFFFF는 32비트 즉시값 비트 패턴으로 해석되어 -1을 나타냅니다.
전체 imm64 이동은 movabsq에서 지원합니다.
조건부 점프와 cmov는 동일한 조건 코드 테이블을 사용합니다.
이 실행기는 교육용 x86-64 부분집합입니다.
```

3 setX(flag 읽기)

setX 명령

- `setX dst`: condition code 조합에 따라 dst의 **최하위 1 byte만** 0 또는 1로 세팅. 나머지 7 byte는 바꾸지 않음 → 위쪽 byte에 남아 있던 값이 그대로 남을 수 있음

명령	조건식	의미
`sete` / `setne`	ZF / ~ZF	Equal·Zero / Not Equal·Not Zero
`sets` / `setns`	SF / ~SF	Negative / Nonnegative
`setg` / `setge`	~(SF^OF) & ~ZF / ~(SF^OF)	Greater / Greater or Equal (signed)
`setl` / `setle`	SF^OF / (SF^OF) | ZF	Less / Less or Equal (signed)
`seta` / `setb`	~CF & ~ZF / CF	Above / Below (unsigned)

- 쉬운 것: sete(cmp로 $a - b$ 계산 → 같으면 0 → ZF=1 → dst = 1), sets(결과가 음수 → SF=1 → dst = 1).

핵심: setl = SF ^ OF

signed less $a < b$ 판정에 SF와 OF를 함께 봄. SF만 보면 안 되는 이유: overflow가 나면 결과의 부호가 뒤집힘.

- 경우 ① overflow 없음 (OF=0) → 결과 부호가 그대로 맞음 → SF=1이면 $a < b$
- 경우 ② overflow 발생 (OF=1) → 부호가 뒤집힘 → SF=0인데도 실제로는 $a < b$ (예: 왼쪽이 아주 작은 음수, 오른쪽이 아주 큰 양수)
- 두 경우를 합치면 $a < b \iff SF \oplus OF$. unsigned 비교는 CF로: $a < b$(below) ⇔ CF=1.

손계산 2 · cmp 4, 2 → setl (수업 예시)

cmpb $4, %al (%al = 2) → $a - b = 2 - 4$, $2 < 4$이므로 setl 결과가 1이어야 함.

2 = 00000010, 4 = 00000100. 뺄셈 = 덧셈 + two's complement: $2 + (-4)$
−4: 4를 반전(11111011) → +1 → 11111100
00000010 + 11111100 = 11111110 (= −2)
flag: ZF = 0 (결과 ≠ 0) · SF = 1 (MSB=1) · OF = 0 (양수−양수는 overflow 불가) · CF = 1 (unsigned 2 < 4 → borrow)

결과

setl = SF ^ OF = 1 ^ 0 = 1 → less 조건 만족 · setb = CF = 1 → unsigned로도 2 < 4

손계산 3 · overflow가 끼는 비교 (8-bit, cmp b, a 기준 a − b)

a, b	8-bit 결과	SF	OF	ZF	CF	setl	판정
a = −2, b = −1	11111111 (−1)	1	0	0	1	1	−2 < −1 ✓
a = −100, b = 100	00111000 (+56)	0	1	0	0	1	−100 < 100 ✓
a = 100, b = −100	11001000 (−56)	1	1	0	1	0	100 > −100 ✓
a = 4, b = 4	00000000	0	0	1	0	0	같음 (sete = 1)

- 두 번째 줄: 진짜 값 −200은 8-bit 범위(−128~127) 밖 → overflow로 +56 → SF=0이지만 OF=1이 보정 → setl = 1. SF만 봤다면 "−100 ≥ 100"이라는 틀린 결론.
- 세 번째 줄: 진짜 값 +200 → overflow로 −56 → SF=1이지만 OF=1 → setl = 0.

```
8bit: cmp $1, a (a = -128 = 10000000)
a - 1 = 01111111 (+127)  SF=0 OF=1 CF=0
setl = SF ^ OF = 1 → -128 < 1 맞게 판정
```

(교수님은 "-256"이라고 말했지만 8bit 최소값은 −128. 원문 교정)

- loop과의 연결: less 조건이 중요한 이유는 반복문: 매 반복마다 i < count인지 비교 → 만족하면 계속, 아니면 탈출. 밑에서는 cmp로 flag를 세팅하고 그 flag 조합으로 조건 유지 여부를 판단.

Byte Register와 movzbl

- setX는 byte 레지스터 하나에 씀. 모든 정수 레지스터는 최하위 byte를 이름으로 참조 가능: %al, %bl … (%rax 64bit, %eax 32bit, %ax 16bit, %ah/%al 상위/하위 8bit)
- **setX하면 앞의 56-bit는 그대로 남음** → %al=1은 되지만 %rax=1은 아님 → 보통 `movzbl` 로 마무리
- **movzbl**: byte → 32-bit zero extension (1바이트 값을 4바이트로 확장하며 나머지를 0으로 채움). 32-bit 명령은 64-bit 레지스터의 상위 32 bit도 0으로 만듦 → %rax 전체가 0 또는 1

- 예: `movzbl %al, %eax` : %rax = 0x123456789ABCDE01 → %rax = 0x0000000000000001

손계산 4 · C → assembly, gt

```
int gt(long x, long y) { return x > y; }
```

```
gt:
    cmpq  %rsi, %rdi  # x - y 로 flag 세팅 (x:y 비교)
    setg  %al         # x > y 이면 %al = 1
    movzbl %al, %eax  # 나머지 비트 0 (%rax 전체)
    ret
```

- %rdi = x, %rsi = y, %rax = 반환값
- x = 5, y = 3: 5 − 3 = 2 → ZF=0, SF=0, OF=0 → setg = ~(0^0) & ~0 = 1 → 반환 1
- x = 3, y = 3: ZF=1 → setg = 0 → 반환 0

4 Conditional Branch

jX 명령

- `jX` : 조건 코드에 따라 다른 위치로 점프 · `jmp` 는 조건 없이 항상 점프 (unconditional)
- je · jne · js · jns · jg · jge · jl · jle · ja · jb: setX와 같은 조건식을 점프에 적용한 것
- signed 비교: jg, jge, jl, jle (SF^OF 사용) · unsigned 비교: ja, jb (CF 사용)

absdiff: 옛날 방식 분기

`gcc -Og -S -fno-if-conversion control.c` . −Og: 최적화를 거의 안 해 C 구조를 그대로 따름 · **−fno-if-conversion**: if/else를 cmov로 바꾸지 말고 분기로 남기라는 플래그

```
long absdiff(long x, long y) {
    long result;
    if (x > y) result = x - y;
    else       result = y - x;
    return result;
}
```

```
absdiff:
    cmpq %rsi, %rdi  # x:y
    jle  .L4         # x <= y 이면 else로
    movq %rdi, %rax
    subq %rsi, %rax  # result = x - y
```

```
    ret
.L4:                    # x <= y
    movq %rsi, %rax
    subq %rdi, %rax  # result = y - x
    ret
```

- x는 %rdi, y는 %rsi, 반환값은 %rax. cmpq %rsi, %rdi → x − y로 플래그 세팅, jle .L4 → x ≤ y면 else 블록으로 점프
- **C 조건과 반대 조건(x > y → jle)으로 else로 점프**
- .L4 같은 label: 코드 안 위치를 사람이 읽기 쉬운 이름으로 붙인 것, 어셈블러가 실제 주소로 바꿔줌

---

goto 버전으로 보기

- goto: label 위치로 바로 점프 → 어셈블리 분기를 C로 추상화한 형태. 컴파일러가 C를 어떻게 점프로 바꾸는지 볼 때 어셈블리와 1:1 대응이라 읽기 쉬움

```
long absdiff_j(long x, long y) {
    long result;
    int ntest = x <= y;
    if (ntest) goto Else;
    result = x - y;
    return result;
Else:
    result = y - x;
    return result;
}
```

일반적인 조건식 변환: val = test ? then_expr : else_expr;

```
    ntest = !(test);
    if (ntest) goto Else;
    val = then_expr;
    goto Done;
Else:
    val = else_expr;
Done:
```

- then과 else를 별도 코드 영역으로 만들고 조건에 맞는 쪽만 실행. absdiff는 각 블록에서 바로 return했지만 실제로는 if/else 뒤에 코드가 이어지므로 then 끝에서 goto Done으로 합류 지점에 점프 (else는 끝나면 그대로 Done으로 흘러감, 점프 불필요)

5 Conditional Move

왜 분기를 피하는가: 파이프라인

- 명령어 하나는 여러 단계로 나뉨 (명령 해석 → 데이터 읽기 → 연산 → 메모리 쓰기)
- 파이프라이닝은 여러 명령어의 서로 다른 실행 단계를 겹쳐 처리하여 처리량을 높인다. 파이프라인 단계 수를 늘리면 클럭 주파수를 높일 가능성이 있지만, 단계 수가 많을수록 항상 성능이 좋아지는 것은 아니다. 분기 예측이 실패하면 잘못된 경로에서 가져온 명령어를 폐기하고 올바른 경로에서 실행을 재개해야 하므로 성능 손실이 발생할 수 있다. 분기 예측이 성공하면 이러한 오예측 페널티는 발생하지 않으며, 컴파일러는 분기와 조건부 이동의 비용을 비교해 적절한 방법을 선택한다. (단계 수는 예전 4~5단계, 지금은 20단계 정도)

파이프라인 5단계 실행 트레이스

5-stage pipeline · 사이클 단위 실행 추적

분기 없음 | 조건 분기 | ← 이전 | 재생 | 다음 → | 처음으로

명령어	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
I1 add R2,R3 → R1	IF	ID	EX	MEM	WB					
I2 xor R5,R6 → R7		IF	ID	EX	MEM	WB				
I3 sub R8,R9 → R10			IF	ID	EX	MEM	WB			
I4 add R11,R12 → R13				IF	ID	EX	MEM	WB		
I5 and R14,R15 → R16					IF	ID	EX	MEM	WB	
I6 or R17,R18 → R19						IF	ID	EX	MEM	WB

현재 C1 — IF: I1 Fetch · ID: — · EX: — · MEM: — · WB: —

누적 사이클: **1 / 10** 완료 명령어: **0 / 6** 누적 CPI: **—** 오예측 페널티: **0 cycles**

IF → ID → EX → MEM → WB 순서로 진행합니다. 파이프라인이 채워진 뒤에는 steady state에서 매 사이클 한 명령어가 완료됩니다.

각 셀은 해당 사이클의 stage입니다. 현재 사이클은 회색 열과 빨간 테두리로 표시합니다.

분기 예측이 틀리면 이미 가져온 명령어를 버려야 하므로 분기 비용이 커질 수 있습니다. 따라서 조건에 따라 컴파일러는 분기 대신 cmov를 선택할 수 있습니다.

데이터 해저드: stall과 forwarding

RAW 의존성 · forwarding 유무에 따른 파이프라인 비교

ALU RAW | load-use | forwarding: 있음

← 이전 | 재생 | 다음 → | 처음으로

명령어	C1	C2	C3	C4	C5	C6
I1 add R2,R3 → R1	IF	ID	EX	MEM	WB	
I2 sub R1,R4 → R5		IF	ID	EX	MEM	WB

현재 C1 — IF: I1 Fetch · ID: — · EX: — · MEM: — · WB: —

누적 사이클: **1 / 6** 완료 명령어: **0 / 2** 누적 CPI: —

forwarding 있음: 6 cycles (CPI 3.00) · forwarding 없음: 8 cycles (CPI 4.00)

I2가 필요로 하는 R1을 I1의 EX 결과에서 바로 forwarding하므로 RAW 의존성 때문에 생기는 stall이 없습니다.

bubble은 EX에 삽입된 빈 단계입니다. ID의 stall은 의존 명령어가 대기 중임을 뜻합니다. 빨간 화살표는 forwarding 경로입니다.

RAW 해저드는 값을 생산하는 명령어와 그 값을 소비하는 명령어 사이의 데이터 의존성 때문에 발생합니다. forwarding은 대기를 줄이지만 load-use에서는 값이 준비되는 시점 때문에 bubble이 남을 수 있습니다.

---

cmov의 아이디어

- **conditional move** (cmovX src, dst): if (test) dst = src를 명령어 하나로 수행
- then과 else를 둘 다 미리 계산해 "양손에 들고" 있다가, 플래그를 보고 하나를 고름 → 점프 없음

```
    result = then_expr;
    eval = else_expr;
    nt = !(test);
    if (nt) result = eval;
    return result;
```

```
absdiff:
    movq %rdi, %rax
    subq %rsi, %rax   # result = x - y
    movq %rsi, %rdx
    subq %rdi, %rdx   # eval = y - x
    cmpq %rsi, %rdi   # x:y
    cmovle %rdx, %rax # x <= y 이면 result = eval
    ret
```

- rax = x − y, rdx = y − x를 다 계산한 뒤 cmpq로 비교 → cmovle: x ≤ y면 rax를 rdx로 덮어씀
- 장점: 점프가 없어 파이프라인 손실 없음 · 단점: 둘 중 하나만 필요한데 둘 다 계산 → then/else 계산이 얼마나 가벼운지에 달림

---

cmov를 쓰면 안 되는 경우: GCC는 안전하다고 확신할 때만 사용

- **비싼 계산**: val = test(x) ? hard1(x) : hard2(x) → 둘 다 계산하므로 손해, 계산이 아주 간단할 때만 이득
- **위험한 계산**: val = p ? *p : 0 → p가 null이어도 *p를 계산해버림 → null pointer 역참조
- **부작용 있는 계산**: val = x > 0 ? x*=7 : x+=3 → 둘 다 실행되면 x가 두 번 바뀌어 결과가 틀림
- 결론: 컴파일러는 성능·안전성·부작용을 다 따져서 필요할 때만 변환

손계산 5 · flag 추적 (슬라이드 Exercise): 각 명령 후 %rax, SF·CF·OF·ZF

명령	%rax	SF	CF	OF	ZF
xorq %rax, %rax	0000 0000 0000 0000	0	0	0	1
subq $1, %rax	FFFF FFFF FFFF FFFF	1	1	0	0
cmpq $2, %rax	FFFF FFFF FFFF FFFF	1	0	0	0
setl %al	FFFF FFFF FFFF FF01	1	0	0	0
movzbl %al, %eax	0000 0000 0000 0001	1	0	0	0

- xorq %rax, %rax → 자기 자신과 XOR = 0 → ZF = 1
- subq $1, %rax → 0 − 1 = −1 = FFFF…FFFF: 아무것도 없는 0에서 빼서 비트가 전부 1 → borrow → CF = 1, 음수 → SF = 1
- cmpq $2, %rax → −1 − 2 = −3 계산 (rax는 안 바뀜): SF = 1, OF = 0 (범위 안), CF = 0 (unsigned로 FFFF…FFFF ≥ 2라 borrow 없음)
- setl %al → SF ^ OF = 1 → al = 01 (−1 < 2 맞음), 나머지 바이트는 그대로 → FFFF…FF01
- movzbl %al, %eax → 상위 비트 전부 0 → rax = 1
- **setl, movzbl은 조건 코드를 바꾸지 않음** → 플래그는 끝까지 SF=1 CF=0 OF=0 ZF=0

6 Loop

do-while

- do-while은 body를 최소 한 번 실행한 뒤 조건 검사, while은 조건부터 검사
- goto 버전: `Loop: body; if (test) goto Loop;`
- 예제 pcount: x의 1 비트 개수 세기 (population count). LSB를 0x1로 마스킹해 result에 누적하고 x를 오른쪽으로 1씩 shift, x가 0이 되면 종료

```
    movl $0, %eax     # result = 0
.L2:                  # loop:
    movq %rdi, %rdx
```

```
    andl $1, %edx     # t = x & 0x1
    addq %rdx, %rax   # result += t
    shrq %rdi         # x >>= 1 (ZF 세팅)
    jne  .L2          # if (x) goto loop
    ret
```

- .L2가 goto 버전의 Loop label에 대응 · shrq %rdi가 ZF를 세팅 → jne: x가 0이 아니면 다시 Loop. goto 버전을 거치면 어셈블리 매핑이 아주 깔끔하게 보임

---

while 변환 1(jump-to-middle, −Og)

```
    goto Test;
Loop:
    body
Test:
    if (test) goto Loop;
```

- 조건을 먼저 검사해야 하므로 처음에 Test로 무조건 점프 → 참이면 Loop로 돌아가 body 실행, 거짓이면 그대로 빠져나감

---

while 변환 2(do-while conversion, −O1)

```
    if (!test) goto Done;
Loop:
    body
    if (test) goto Loop;
Done:
```

- do-while 앞에 조건 검사(guard)를 하나 붙인 형태 → 처음부터 거짓이면 Done으로 바로 빠짐
- 왜 −O1은 이 방식인가: jump-to-middle은 x ≠ 0일 때 첫 body 전에 Test로 한 번, Loop로 다시 한 번 → 불필요한 점프 2번. do-while 변환은 guard 통과 후 점프 없이 바로 body로. 점프는 파이프라인을 깨므로 적을수록 좋음

---

for → while → do-while

- `for (init; test; update)` body는 기본적으로 `init; while (test) { body; update; }` 형태로 대응시킬 수 있다. 다만 body 안에 continue가 있으면 for에서는 update를 실행한 뒤 조건을 검사하지만, 단순한

while 변환에서는 update를 건너뛸 수 있다. 따라서 완전히 동일한 동작을 유지하려면 continue 경로에서도 update가 실행되도록 변환해야 한다.

- 컴파일 순서: for → while → do-while 변환 → goto (어셈블리 구조 그대로)
- 예제 pcount_for: WSIZE = 8×sizeof(long) = 64, (x >> i) & 0x1로 i번째 비트 확인. −O1에서는 처음 guard가 사라짐. i = 0이고 WSIZE = 64라 첫 검사는 무조건 참 (x = 0이어도 각 비트가 0이라 result가 안 바뀔 뿐 정확성에 문제 없음)

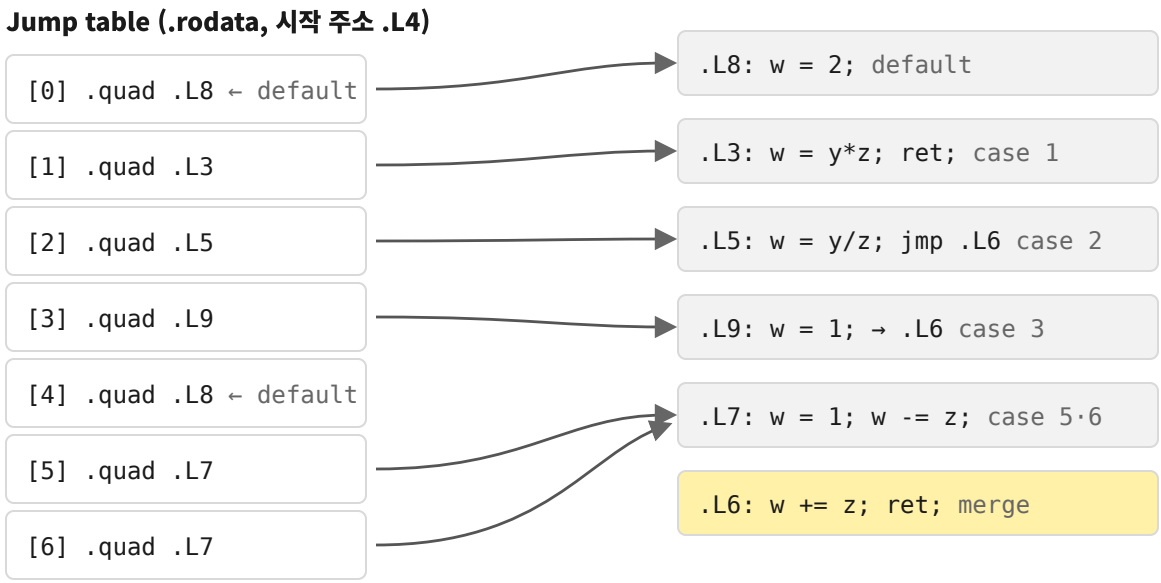
7 Switch 문

switch_eg 예제

```
long switch_eg(long x, long y, long z) {
    long w = 1;
    switch (x) {
    case 1:  w = y*z; break;
    case 2:  w = y/z;        // fall through
    case 3:  w += z;  break;
    case 5:
    case 6:  w -= z;  break;
    default: w = 2;
    }
    return w;
}
```

- 여러 case label: case 5, 6이 같은 코드 실행 (특수한 fall through) · fall through: case 2는 break가 없어 case 3까지 실행 · 빠진 case: 4, 그리고 7 이상은 전부 default
- if-else를 중첩해서 같은 동작을 만들 수 있지만, switch는 **jump table**로 다르게 처리

jump table 구조



jump table = 각 case 코드 블록의 주소를 담은 배열. index = x 값 → JTab[x]가 해당 블록의 시작 주소 (슬라이드 그림 재구성)

- 각 case 코드 블록의 시작 위치에 label(Targ0, Targ1 …)을 붙이고 jump table은 그 label들의 주소를 담은 배열 → `goto *JTab[x];` 한 번 조회로 바로 점프, 조건을 하나씩 비교할 필요 없음

```
    .section .rodata
    .align 8
.L4:
    .quad .L8   # x = 0  → default
    .quad .L3   # x = 1
    .quad .L5   # x = 2
    .quad .L9   # x = 3
    .quad .L8   # x = 4  → default (빠진 case)
    .quad .L7   # x = 5
    .quad .L7   # x = 6
```

- `.section .rodata` → 읽기 전용 영역, .L4가 테이블 시작 주소 · `.quad` → 엔트리 하나가 8바이트 주소 (주소는 64bit)
- 테이블에 저장된 것은 "L3"라는 문자열이 아니라 그 label의 **실제 메모리 주소**. 바이너리에서는 `gdb x/8xg <테이블 주소>` 로 확인 가능

direct jump와 indirect jump

- direct: `jmp .L8` → label 위치로 바로 점프

- indirect: `jmp *.L4(,%rdi,8)` → 메모리에 저장된 주소를 읽어 그곳으로 점프. *는 pointer 역참조처럼 "이 주소에 들어 있는 값으로 가라"
- 주소 계산은 D(Rb, Ri, S) 그대로: base 없음, displacement = .L4, index = x, scale = 8 → **.L4 + 8x 위치의 8바이트를 읽어 점프**, 0 ≤ x ≤ 6일 때만 유효

---

setup 코드

```
switch_eg:
    movq %rdx, %rcx       # z를 %rcx로 복사
    cmpq $6, %rdi         # x:6
    ja   .L8              # unsigned로 x > 6 (음수 포함) → default
    jmp  *.L4(,%rdi,8)    # goto *JTab[x]
```

- movq %rdx, %rcx: (보충) case 2의 idivq가 %rdx:%rax를 피제수로 쓰고 cqto가 %rdx를 덮어쓰므로 z를 미리 옮겨 둠
- cmpq $6 + ja: ja는 unsigned 비교라 x가 음수여도 아주 큰 수로 보여 default로 감 → 범위 검사 한 번으로 해결
- w = 1 초기화가 여기 없음: w를 덮어쓰는 case(1, 2, default)는 초기값 불필요, w에 더하거나 빼는 case(3, 5, 6)만 필요 → 컴파일러가 필요한 case 안에서만 w = 1을 넣음

각 case 블록

```
.L3:                  # case 1: w = y*z
    movq %rsi, %rax   # ret = y
    imulq %rdx, %rax  # ret *= z
    ret               # break → 함수 끝
```

- fall through 처리 (case 2, 3): C로 풀면 case 2는 w = y/z; goto Merge; · case 3은 w = 1; Merge: w += z;
  - .L5 (case 2): y/z 계산 후 `jmp .L6` (merge)로 case 3의 w += z 부분에 합류. case 2는 w를 덮어쓰므로 w = 1 불필요
  - .L9 (case 3): `movl $1, %eax` 로 w = 1을 여기서 초기화한 뒤 .L6으로 흘러감
- .L7 (case 5, 6): `movl $1, %eax; subq %rcx, %rax; ret` . case 5와 case 6은 이 블록에서 반환하므로 .L6의 w += z 블록으로 이어지지 않는다. 여기서 %rcx는 앞서 복사해 둔 z를 담는다. 테이블에서 x=5, x=6 엔트리가 둘 다 .L7 · .L8 (default): w = 2

정리와 jump table의 다른 쓰임

C의 for, switch 문법은 사람이 읽기 쉽지만 어셈블리에는 그런 문법이 없음 → 기본 명령어 + 조건 코드 + 점프로 구현. loop은 do-while 또는 jump-to-middle로, 큰 switch는 jump table로, 드문드문한(sparse) switch는 decision tree(if-else 연쇄)로 변환될 수 있음.

8 시험 포인트

- CF는 unsigned overflow(carry/borrow), OF는 signed overflow
- SF는 결과 MSB, ZF는 결과 0
- add·sub·cmp는 CF와 OF를 둘 다 계산해 세팅 → signed/unsigned 중 어떤 비교인지는 쓰는 jCC가 정함 (명령마다 갱신 규칙은 다름: leaq 미갱신, inc/dec는 CF 보존, 논리 명령은 CF=OF=0)
- leaq는 flag를 바꾸지 않음
- 뺄셈의 CF = borrow 여부: 실제 carry out이 없으면 CF = 1 (unsigned로 a < b)
- OF 조건: 덧셈은 같은 부호끼리 더해 부호가 바뀔 때
- 뺄셈은 다른 부호끼리 빼서 결과 부호가 a와 다를 때
- cmpq b, a는 $a - b$ (순서 주의), 결과를 저장하지 않음
- testq b, a는 $a \,\&\, b$ (결과 저장 안 함), CF·OF는 항상 0, testq %rax, %rax로 0/음수 검사
- setl = SF^OF (jl도 같음: 가장 작은 음수에서 빼서 overflow가 나도 대소 판정이 맞음), setle = (SF^OF)|ZF, setg = ~(SF^OF)&~ZF, setb = CF, seta = ~CF&~ZF
- setX는 최하위 1바이트만 씀 → movzbl로 상위를 0으로 만들어야 함
- 32bit 목적지 명령은 상위 32bit를 자동으로 0으로
- 같은 비트열이라도 signed/unsigned 비교 결과가 다를 수 있음 (−1 vs 2)
- 조건 분기는 C 조건의 반대 조건으로 jump
- −fno-if-conversion은 cmov 대신 분기로 남기라는 옵션
- cmov는 then/else를 둘 다 계산
- 비싼 계산은 양쪽 식을 모두 계산하게 되어 성능상 불리할 수 있으므로 분기와 조건부 이동의 비용을 비교해야 한다. 한쪽 식을 무조건 계산하면 null 역참조 위험이 발생하거나 부작용 때문에 프로그램의 의미가 달라지는 경우에는 단순한 cmov 변환을 사용하지 않아야 한다.
- 분기가 비싼 이유: 파이프라인에 미리 넣은 명령을 버려야 함
- 이 예제의 GCC 출력에서는 -Og가 jump-to-middle 형태를, -O1이 guard를 둔 do-while conversion 형태를 사용한다. 이는 해당 코드와 컴파일러 환경에서 관찰한 결과이며, 최적화 옵션만으로 모든 while문이 항상 같은 형태로 변환된다고 일반화할 수는 없다.

- for는 init / while(test) { body; update; }로 바꾼 뒤 같은 방식으로 컴파일
- 처음 guard는 상수 비교면 제거됨
- jump table: 엔트리 8바이트, jmp *.L4(,%rdi,8) = .L4 + 8x 주소의 값을 읽어 점프
- cmpq $6 + ja 한 번으로 x > 6과 음수 x를 모두 default로 보냄 (unsigned 비교)
- 빠진 case 값은 테이블에서 default label을 가리킴
- 같은 코드의 case는 같은 label을 가리킴
- fall through는 jmp로 다음 case의 중간(merge)에 합류하는 방식
- w = 1 같은 초기화는 그 값이 필요한 case 안으로 옮겨질 수 있음

9 연습문제

문제 1. 8-bit에서 addb로 $(-128) + (-1)$을 계산할 때 결과 비트와 CF, ZF, SF, OF를 구하라.

정답 보기

−128 = 10000000, −1 = 11111111
합 = 1 01111111 → 8-bit 결과 01111111 = 127
CF = 1 (MSB에서 carry out) · ZF = 0 · SF = 0
OF = 1 (음수 + 음수 = 양수)

결과 **결과 127, CF=1 ZF=0 SF=0 OF=1**

문제 2. 8-bit에서 cmpb $1, %al (%al = 0x80) (−128)일 때 SF, OF, ZF, CF와 setl, setb의 값을 구하라.

정답 보기

a − b = −128 − 1 = −129 → 범위 밖 → 8-bit 결과 01111111 = 127
SF = 0, ZF = 0 · OF = 1 (a < 0, b > 0인데 결과 > 0) · CF = 0 (unsigned 128 ≥ 1, borrow 없음)
setl = SF^OF = 1 → signed로 −128 < 1 맞음 · setb = CF = 0 → unsigned로 128 < 1 아님

결과 **SF=0 OF=1 ZF=0 CF=0, setl=1, setb=0**

문제 3. testq %rax, %rax 실행 후 %rax = 0x8000000000000000이라면 ZF, SF, CF, OF와 sets, sete의 결과는?

정답 보기

%rax & %rax = %rax 그대로 → MSB = 1, 0 아님

ZF = 0, SF = 1, CF = 0, OF = 0 (test는 항상 clear)

결과 **sets = 1 (음수), sete = 0**

문제 4. %rax = 0x1234567890ABCDEF인 상태에서 cmpq %rax, %rax → sete %al을 실행하면 %rax는? 이어서 movzbl %al, %eax 후에는?

정답 보기

---

cmp: a − a = 0 → ZF = 1 → sete가 %al = 0x01

sete 후: 상위 7 byte 유지 → %rax = 0x1234567890ABCD01

movzbl 후: %eax = 1, 상위 32 bit도 0 → %rax = 0x0000000000000001

결과 **0x1234567890ABCD01 → 0x0000000000000001**

문제 5. 다음 C 함수를 cmp와 setX, movzbl로 번역하라. (x는 %rdi, y는 %rsi, 반환 %rax)

```
int le_u(unsigned long x, unsigned long y) { return x <= y; }
```

정답 보기

---

unsigned이므로 CF 기반 조건 사용: x <= y ⇔ ~(x > y) ⇔ ~(~CF & ~ZF) = CF | ZF → setbe (below or equal)

cmpq %rsi, %rdi → x − y로 flag 세팅

setbe %al · movzbl %al, %eax · ret

결과

cmpq %rsi,%rdi / setbe %al / movzbl %al,%eax / ret. signed였다면 setle 사용

문제 6. %rdi = −1, %rsi = 1일 때 cmpq %rsi, %rdi 직후 jl, jb, ja 중 점프하는 것은?

정답 보기

−1 − 1 = −2 → SF = 1, OF = 0, unsigned로 FFFF⋯FFFF ≥ 1이라 CF = 0, ZF = 0
jl: SF ^ OF = 1 → 점프 (signed로 −1 < 1) · jb: CF = 0 → 점프 안 함 · ja: ~CF & ~ZF = 1 → 점프 (unsigned로 FFFF⋯FFFF > 1)

결과 **jl과 ja는 점프, jb는 점프 안 함**

문제 7. switch_eg(2, 7, 2)와 switch_eg(−1, 7, 2)의 반환값과 거쳐 가는 label은?

정답 보기

x = 2: jmp *.L4(,%rdi,8) → .L5, w = 7/2 = 3, jmp .L6에서 w += 2 → 5
x = −1: cmpq $6 후 ja가 unsigned 비교라 −1 = FFFF⋯FFFF > 6 → .L8 (default) → 2

결과 **switch_eg(2,7,2) = 5 (.L5→.L6), switch_eg(−1,7,2) = 2 (.L8)**

문제 8. 다음 중 컴파일러가 cmov로 바꾸면 안 되는 것과 이유는?
① val = x > y ? x − y : y − x; ② val = p ? *p : 0; ③ val = x > 0 ? x *= 7 : x += 3;

정답 보기

① 괜찮음: 둘 다 가벼운 뺄셈
② 안 됨: p가 null이어도 *p를 계산해 null pointer 역참조
③ 안 됨: 두 식이 다 실행되어 x가 두 번 바뀜 (부작용)

결과 **② (null 역참조 위험), ③ (부작용). ①은 가능**

문제 9. while (x) { result += x & 1; x >>= 1; }를 jump-to-middle과 do-while conversion 두 가지 goto 형태로 쓰고, x가 0이 아닐 때 첫 body 전까지 실제로 일어나는 점프 횟수를 비교하라.

정답 보기

jump-to-middle: goto Test; Loop: body; Test: if (x) goto Loop;
do-while conversion: if (!x) goto Done; Loop: body; if (x) goto Loop; Done:
x ≠ 0일 때 jump-to-middle은 Test로 1번, Loop로 1번 → 2번 · do-while conversion은 guard가 거짓이라 점프 없이 바로 body → 0번

결과 **jump-to-middle 2번 vs do-while conversion 0번**

문제 10. case가 0, 1, 2, 3, 1000뿐인 switch는 jump table로 만들기에 적합한가?

정답 보기

적합하지 않음: 테이블이 x 값으로 인덱싱되므로 1000까지 엔트리(8바이트씩)가 필요하고 대부분 default
sparse한 switch는 if-elseif 형태의 decision tree로 컴파일될 수 있음

결과 **부적합. decision tree가 나음**